International Journal of Information Security (2025) 24:204
https://doi.org/10.1007/s10207-025-01124-x

REGULAR CONTRIBUTION

Flow Hijacking in eBPF: Exploitation and Mitigation Across both Interpreter and JIT Execution

Yifei Wu[1] · Qirui Liu[1] · Wenbo Shen[1] · Zhuoruo Zhang[1] · Jiayi Hu[1] · Rui Chang[1]

Received: 30 June 2025 / Accepted: 29 August 2025 / Published online: 9 September 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract
The extended Berkeley Packet Filter (eBPF) has been widely used and is regarded as a secure alternative to kernel modules for extending kernel functionality. As an emerging kernel subsystem, it is expected to remain resilient against exploitation through existing kernel vulnerabilities. Unfortunately, the exploitability of eBPF has not been thoroughly studied. In this paper, we systematically investigate the security of eBPF and identify two new risks: the absence of hijacking defenses in the eBPF interpreter and incomplete Control Flow Integrity (CFI) enforcement in the Just-In-Time (JIT) compiler. To demonstrate practical impact, we introduce Flow Hijacking, a generic eBPF exploitation technique that bypasses kernel code and CFI protections to hijack the eBPF execution. We propose two key techniques: Tailcall Trampoline, which corrupts tailcalls to redirect the interpreter to injected code without modifying function pointers, and JIT-Type Confusion, which forges trusted pointers in JITed code to gain arbitrary kernel memory access. We further formulate CVE requirements, evaluate Flow Hijacking against 16 real-world CVEs from different kernel subsystems, and show that it consistently amplifies their capabilities to bypass kernel protections. Additionally, we present a formal proof showing that Flow Hijacking achieves Turing completeness when targeting the interpreter. Finally, we develop a lightweight runtime validation mechanism that effectively mitigates Flow Hijacking with less than 1% performance overhead. We are communicating with the Linux community to address the identified issues.

Keywords eBPF · Interpreter · JIT · Bug Exploitation · Linux Kernel

1 Introduction

Extended Berkeley Packet Filter (eBPF) is a novel technique allowing users to load and execute customized programs in the Linux kernel. Different from the traditional way of extending kernel functionality via kernel modules, eBPF is endowed with a verifier designed to ascertain the safety and integrity of eBPF programs prior to their execution within the kernel space. With this security guarantee, eBPF is regarded as a much safer alternative to traditional kernel modules. Nowadays, eBPF is emerging as the predominant method for extending kernel functionality, steadily supplanting the traditional use of kernel modules. It has been widely adopted in many domains[1], such as kernel debugging and tracing [1], network performance boosting [2], cloud-native runtime monitoring [3], and security enforcement [4].

Existing security efforts on eBPF have primarily focused on preventing malicious eBPF-to-kernel interactions. Specifically, eBPF heavily relies on its verifier for detecting malicious eBPF programs, preventing them from attacking the kernel. Therefore, the correctness of the verifier is critical to the security of eBPF. Researchers have conducted various works to eliminate verifier bugs [5, 6], improve its design to

✉ Wenbo Shen
shenwenbo@zju.edu.cn

Yifei Wu
yinwhe@zju.edu.cn

Qirui Liu
liuqirui@zju.edu.cn

Zhuoruo Zhang
zhangzhuoruo@zju.edu.cn

Jiayi Hu
hujiayi@zju.edu.cn

Rui Chang
crix1021@zju.edu.cn

[1] Zhejiang University, Hangzhou, China

[1] https://ebpf.io/applications/

reduce complexity [7, 8], and formally verify its logic and behavior [9].

However, solely preventing eBPF-to-kernel attacks is insufficient: eBPF should also not become an abusive component exploited by existing kernel vulnerabilities. Particularly, given the massive code base and complexity of the Linux kernel, such vulnerabilities are inevitable. Therefore, as a new kernel subsystem, eBPF must consider these kernel vulnerabilities when designing its security model. *Specifically, eBPF should not be exploited by these vulnerabilities to attack the kernel, such as bypassing kernel protection.* Unfortunately, existing research on eBPF security has predominantly focused on enhancing the verifier. While several blogs have demonstrated attacks that corrupt function pointers to trigger malicious eBPF bytecode [10, 11], these approaches are ad-hoc and rely on the assumption of compromised Control Flow Integrity (CFI). Therefore, the expressiveness of eBPF exploitation has not been fully investigated so far.

To fill this gap, we conduct a systematic study on eBPF exploitability and uncover two previously unidentified attack vectors: *eBPF interpreter lacks defenses against hijacking, and the Just-In-Time (JIT) compiler's CFI enforcement is incomplete.* Building on this insight, we propose a novel exploitation approach named *Flow Hijacking*, which bypasses kernel code/CFI protections, achieving arbitrary eBPF bytecode execution on the interpreter path, and arbitrary kernel memory access on the JIT path. To launch Flow Hijacking across both the eBPF interpreter and JIT execution, we face two main challenges.

First, for the interpreter execution, corrupting the eBPF execution flow while preserving kernel CFI is non-trivial. In this execution model, the entire eBPF program data structure—including the bytecode—is marked read-only as a security hardening measure. As a result, attackers cannot simply exploit kernel bugs to overwrite this code directly. Furthermore, attacks that rely on corrupting kernel pointers [10, 11] are now thwarted by kernel CFI, making it infeasible to hijack control flow by tampering with kernel pointers.

Second, for JIT execution, crafting malicious code is particularly challenging. All JITed code resides in W⊕X-protected pages in kernel space, and mechanisms like Supervisor Mode Access Prevention (SMEP) prevent the kernel from executing user-space code, rendering direct malicious code injection impossible. Additionally, since eBPF programs are strictly verified before execution, it is difficult to craft the precise stack or register states needed for conventional code reuse attacks.

To overcome the above challenges, we propose two novel exploitation techniques. *Tailcall Trampoline* tampers with heap-resident `prog-array` maps and abuses the tailcall mechanism to redirect execution to attacker-injected bytecode. When the JIT is enabled, direct code injection is infeasible; instead, *JIT-Type Confusion* skips the critical verified code and tricks eBPF into using attacker-controlled data as a kernel pointer, thereby achieving arbitrary kernel memory access.

To demonstrate the exploitability of Flow Hijacking, we formulate the requirements for applicable CVEs and provide techniques for pivoting double-free, out-of-bounds, and use-after-free to exploit our attack. We then validate Flow Hijacking on 16 real-world CVEs originating from various subsystems of the Linux kernel. The results show that it can enhance the capabilities of all these CVEs in bypassing kernel protection. Additionally, to understand the expressiveness of Flow Hijacking when targeting the interpreter, we present a formal proof to establish its Turing completeness.

Finally, to defend against such attacks, we design and implement a runtime validation mechanism within both the eBPF interpreter and JIT execution paths. This defense verifies the integrity of executed programs by checking whether the target addresses match those of previously verified ones, effectively blocking any malicious flow hijacks across both paths and introducing less than 1% performance overhead. Moreover, we have reported our findings and submitted the mitigation patch via the Linux mailing lists, and the maintainers have confirmed the issue. Currently, we are still actively communicating with the Linux community to address the identified issues and improve the overall security of eBPF and the Linux kernel.

This paper makes the following contributions:

- **New attack vector.** We reveal previously unknown weaknesses in both the eBPF interpreter and JIT that permit execution flow hijacking.
- **New exploitation technique.** We present *Flow Hijacking*, including *Tailcall Trampoline* and *JIT-Type Confusion*, to bypass kernel code and CFI protections, thereby achieving arbitrary bytecode execution and arbitrary kernel memory access.
- **Exploitability evaluation.** We evaluate the exploitability of Flow Hijacking using 16 CVEs across multiple subsystems and demonstrate that it can enhance the capabilities of all these exploits.
- **Expressiveness proof.** We provide formal proof to demonstrate that Flow Hijacking is Turing-complete when targeting the interpreter, thereby showing its strong expressiveness.
- **Defense solution.** We implement an IDR-based runtime validation mechanism to defeat Flow Hijacking across both eBPF interpreter and JIT execution paths, with less than 1% performance overhead.

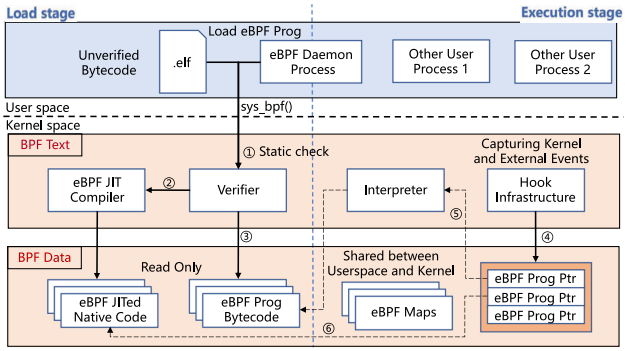


Fig. 1 eBPF workflow is split into load and execution stages

2 eBPF Background

2.1 eBPF Workflow

As depicted in Figure 1, the eBPF workflow can be divided into the load stage and the execution stage. To deploy eBPF, users typically provide both an eBPF program (in the form of a bytecode-encoded `.elf` file) and a userspace daemon. We use the term eBPF program to refer to the `bpf_prog` structure in the kernel, which contains the bytecode and some other metadata.

Load stage. During the load stage, a user first executes an eBPF daemon program, which invokes the `sys_bpf()` system call to load the unverified eBPF program (bytecode) into the kernel. The kernel then calls the verifier function `bpf_check()` to statically analyze the bytecode (①), ensuring its safety and correctness. Note that such checks are static and only performed once. Programs that fail the verification are rejected with diagnostic logs returned to the user. For programs that pass the check, the kernel will decide whether it should be executed via the interpreter or JIT compiled into native code. For those executed in native code, the kernel further calls the JIT compiler for the corresponding architecture to compile the bytecode into executable machine code (②). The other ones executed via interpreters do not need to go through the JIT compiler (③). All verified eBPF programs are then attached to specific kernel hook points and wait to be triggered.

Execution stage. In the execution stage, eBPF programs are triggered by corresponding kernel or external events, such as system call invocations or incoming network packets. Upon receiving an event (④), the kernel handles the interrupt and turns to execute the attached eBPF programs. If the program is JITed, the kernel directly jumps to the entry of the native code region (⑤). Otherwise, the interpreter function `___bpf_prog_run` is invoked to simulate execution by interpreting the bytecode instructions sequentially (⑥).

2.2 eBPF Programs and Maps

eBPF programs and maps are the main data structures of an eBPF, which are associated with files and accessed by file descriptors. The kernel uses `bpf_prog` structure to represent an eBPF program, which contains the bytecode in its `insnsi` field and a function pointer `bpf_func`. This pointer is set after verification to either the interpreter function (`__bpf_prog_run`) or the entry of JITed native code. To guard against tampering, the entire `bpf_prog` structure, including the bytecode, is marked read-only after verification, and the JITed native code is protected by W⊕X policy.

eBPF maps are used to share data between eBPF programs and userspace. Internally, the eBPF map is a collective name of various eBPF data structures, such as `bpf_array`, `bpf_hash`, and `bpf_ring_buffer`, etc. All these data structures contain a `bpf_map` structure as a field to record general eBPF map metadata and a type-specific data area to store values. Among these, `bpf_array` can be used as either a general array map or a prog-array map, depending on the `map_type` field. While array maps store values, prog-array maps store pointers to `bpf_prog` structures. Note that these pointers reference kernel objects and are not accessible from userspace.

2.3 Existing eBPF Hardening

Bytecode read-only protection. To prevent unauthorized modification of eBPF bytecode, the kernel marks the memory region containing verified code as read-only. Specifically, after an eBPF program is successfully verified by the verifier, the kernel invokes `bpf_prog_select_runtime` to determine its execution mode. If the program is designated for interpretation, `bpf_prog_lock_ro` is called to mark the entire `bpf_prog` structure as read-only, including its `insnsi` field, which stores the bytecode instructions.

JIT code protections. The kernel enforces a strict defense on eBPF JITed code as kernel text. The generated native code is placed in the kernel module mapping region and its page table entries (PTEs) are marked read-only, enforcing the W⊕X policy. In addition, to defend against JIT spraying attacks [12], the kernel introduces `constant blinding` [13], a mitigation that replaces constant values with dynamically computed equivalents. This reduces the likelihood that attacker-controlled immediate values will appear in the emitted machine code, making reliable payload construction more difficult.

3 Threat Model and Methodology Overview

We introduce Flow Hijacking, a technique that exploits vulnerabilities in both the eBPF interpreter and its JIT compiler. This section first clarifies the threat model and assumptions. Next, it gives an overview of the paper's methodology, including both attacks and defenses.

3.1 Threat Model and Assumptions

Attacker's capabilities. We assume the attacker is an unprivileged user without any special kernel capabilities. Specifically, the attacker can access unprivileged eBPF features, such as loading `SOCKET_FILTER` type eBPF programs and creating array map or prog-array map [14], which aligns with assumptions in related works [15, 16]. This assumption is reasonable as we find that, before Linux v5.16, the default configurations for all architectures (e.g., `x86_64` and `arm64`) do not set `CONFIG_BPF_UNPRIV_DEFAULT_OFF`. By default, most of the in-service LTS Linux versions (4.19-5.15) still enable unprivileged eBPF usage. Here, whether the eBPF interpreter or JIT is enabled as the default execution engine is not important, because we can attack both of them.

We further assume that the Linux kernel contains common vulnerabilities allowing attackers to corrupt heap memory. This is a practical assumption as hundreds of such bugs are reported every year, and this number is still increasing, according to NVD. Moreover, this assumption is also widely adopted by previous research on kernel exploitation works [17–19].

Kernel protection deployed. We assume the Linux kernel adopts mainstream protection, including (1) kernel code protection (W⊕X) to prevent kernel code from being manipulated. (2) Supervisor Mode Access Prevention (SMEP) to prohibit the kernel from jumping to userspace code. (3) CFI to protect function pointers. Also, SMAP is enabled to prohibit the kernel from accessing userspace data directly. Moreover, we assume that the page tables in the kernel are protected using existing techniques, such as TZ-RKP [17]. As a result, the attacker cannot corrupt the eBPF bytecode or the JITed code (mapped as read-only). In addition, we assume the kernel has enabled KASLR, making it harder for attackers to infer the memory layout.

Additional JIT Assumption. For the JIT code attack, we further assume that Intel Indirect Branch Tracking (IBT) protection is disabled on x86 kernels. This assumption is valid and reasonable for two reasons. First, the Linux kernel did not enable the IBT feature by default before v6.2. Second, current mainstream distributions also do not enable IBT by default. For example, Ubuntu turns off IBT due to module compatibility issues [20].

3.2 Methodology Overview

Our methodology forms a closed loop that spans vulnerability discovery, exploitation, evaluation, and mitigation. We first define the threat model (Section 3.1). Next, we split the eBPF runtime into the *interpreter* and *JIT* paths and systematically investigate kernel objects that hold pointers to verified eBPF programs. The investigation reveals a critical gap: tailcall instructions in both execution paths perform no runtime validation of their target.

Building on this insight, we develop two complementary attacks. *Tailcall Trampoline* (Section 4) corrupts a heap-resident `prog_array` entry to redirect the interpreter to attack-ersupplied bytecode, while *JIT-Type Confusion* (Section 5) reuses that corruption to reinterpret an attacker-controlled scalar as a trusted kernel pointer inside JITed code, all without violating kernel code/CFI protections. To evaluate the broad applicability of our attacks, we select 16 real-world CVEs from different subsystems and systematically analyze their reachability and exploitability (Section 6). We then prove that, when targeting interpreter, Flow Hijacking attains Turing-completeness (Section 7). The JIT path lacks this property due to W⊕X and SMEP constraints.

Finally, we present a lightweight mitigation (Section 8): an IDR-based runtime validator that checks every tailcall target against a radix-tree of previously verified program IDs and aborts on any mismatch. This patch adds less than 1% in throughput and latency micro-benchmarks.

4 eBPF Interpret Flow Hijacking

This section details how Flow Hijacking compromises the eBPF interpreter via the Tailcall Trampoline technique. The JIT variant follows in Section 5.

4.1 Attack Overview

The basic idea of Flow Hijacking targeting the eBPF interpreter, as shown by Figure 2, is to inject eBPF bytecode as data to bypass the verifier's checks and redirect the interpreter to the attacker-injected bytecode. Our key observation, as summarized in Table 1, is that the current eBPF interpreter lacks defenses against code injection and interpret flow hijack, and thus assists existing CVEs in bypassing kernel protections and achieving arbitrary code execution. Specifically, the eBPF interpreter lacks protection on its interpretation flow, which can be maliciously redirected. And with no mechanism against bytecode injection, an attacker can inject bytecode as data and hijack the interpreter with it, achieving arbitrary code execution.

Though the idea is straightforward, Flow Hijacking must resolve two challenges. First, the eBPF bytecode is read-only

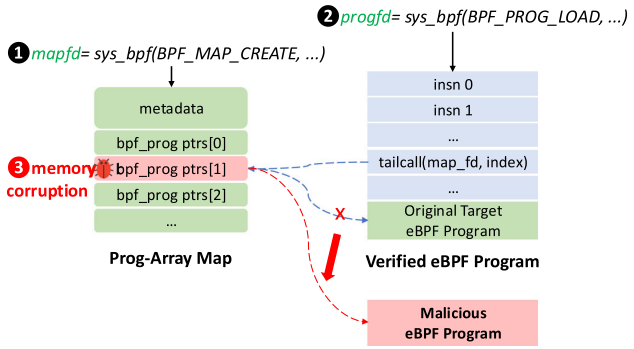


Fig. 2 The basic idea and attacking steps of Flow Hijacking targeting interpreter. It exploits kernel vulnerabilities to redirect the interpreter to execute malicious eBPF bytecode. The attacking steps of ❶ and ❷ don't require any privileges or capabilities

```
1  static u64 ___bpf_prog_run(u64 *regs, const struct
   ↪  bpf_insn *insn)
2  {   ...
3  select_insn:
4      goto *jumptable[insn->code];
5      ...
6      JMP_TAIL_CALL: {
7          struct bpf_map *map = (struct bpf_map *)
           ↪  (unsigned long) BPF_R2;
8          struct bpf_array *array = container_of(map,
           ↪  struct bpf_array, map);
9          struct bpf_prog *prog;
10         u32 index = BPF_R3;
11         ...
12         prog = READ_ONCE(array->ptrs[index]);
13         if (!prog)
14             goto out;
15         insn = prog->insnsi;
16         goto select_insn;
17 out:
18         CONT;}
19   ...}
```

Fig. 3 Tailcall implementations in Interpreter execution

protected and cannot be corrupted. The bytecode is placed in the `bpf_prog` structure, and the whole structure (including bytecode) is marked as read-only after loading. As a result, one cannot overwrite the bytecode directly. Second, it is challenging to trigger malicious bytecode without altering the control flow. Normally, eBPF programs are executed via invoking the function pointer `bpf_prog->bpf_func`. If we trigger a forged eBPF program in this way, such an invocation operation will be blocked by CFI due to corrupted function pointers.

To resolve the above challenges, we propose a novel technique called *Tailcall Trampoline* to leverage existing CVEs to corrupt tailcall program structure and redirect the eBPF interpretation flow to arbitrary bytecode without violating CFI.

4.2 Tailcall Trampoline

As discussed previously, verified eBPF bytecode is marked read-only to prevent direct corruption. However, we notice some other kernel data structures still hold unprotected pointers to `bpf_prog` instances. Motivated by this, we systematically analyzed all usages of `bpf_prog` structure and its pointers, and found that eBPF tailcall loads a `bpf_prog` pointer from a prog-array map allocated on the heap.

As depicted in Figure 3, when the interpreter encounters a tailcall instruction (Line 6), it acquires a pointer to the `bpf_prog` structure of the sub-program by loading the field in the prog-array map's data area with a specified index (Line 12). Next, the interpreter skips the remainder of the main program and jumps into the subprogram for interpretation (Line 15).

Unfortunately, such a design can be exploited by kernel CVEs to hijack the interpretation flow. First, prog-array maps containing `bpf_prog` pointers are heap-allocated with sizes fully controlled by unprivileged users, require no special capabilities to create, and persist for the program's lifetime, making them ideal targets for kernel memory bugs without any timing constraints. Second, the interpreter's tailcall handling never invokes a function pointer, so by corrupting only non-control data in a prog-array entry, an attacker can hijack the interpretation flow without violating CFI.

Attacking steps. Let's use Figure 2 to illustrate the attacking steps of Tailcall Trampoline. First, the attacker needs to create a prog-array by invoking `sys_bpf(BPF_MAP_CREATE, uattr, ...)` with `uattr->map_type` set to `BPF_MAP_TYPE_PROG_ARRAY`, as shown by ❶. This creates a prog-array and returns its file descriptor `mapfd`. Next, a verifier-approved eBPF program is loaded via `sys_bpf(BPF_PROG_LOAD, ...)` ❷, in which the attacker passes `mapfd` to the tailcall so that it loads pointers (i.e., `bpf_prog ptrs[x]`) from `mapfd`'s data area. The above two steps do not require any privileges or capabilities. Note that until now, the interpretation flow is correct. If the eBPF program is triggered, it will follow the correct execution path, as shown by the blue line in Figure 2.

Table 1 Defense comparison between kernel code and eBPF

Attacks	Kernel code	eBPF Interpreter	eBPF JIT
Corrupt existing code	Read-only	Read-only	Read-only
Inject new code	SMEP	**Missing**	SMEP
Hijack execution flow	CFI	**Missing**	**Unsound**

To hijack the bytecode flow, the attacker performs the third step to overwrite `bpf_prog ptrs[x]` to a malicious value by triggering a memory corruption bug (❸ of Figure 2). When the program is triggered again, the interpreter loads the corrupted pointer (Line 12 of Figure 3) and begins interpreting the injected bytecode, as shown by the red line in Figure 2. Since bytecode is stored as data in the kernel, malicious bytecode injection can be done via `copy_from_user` or crafted network packets, bypassing the read-only protection of the original `bpf_prog` region. Here we omit the discussion for brevity.

With the above steps, Tailcall Trampoline redirects the tailcall bytecode to any bytecode successfully. Moreover, Tailcall Trampoline does not modify any function pointers or return addresses and thus can bypass CFI totally. To summarize, with Tailcall Trampoline, Flow Hijacking can bypass the eBPF verifier and execute any bytecode.

5 eBPF JIT Flow Hijacking

In this section, we extend Flow Hijacking of JIT execution. Particularly, we propose the JIT-Type Confusion technique to tackle challenges unique to JIT environments.

5.1 Attack Overview

To attack the eBPF JIT, our key observation is that although recent kernels have integrated kernel CFI into the eBPF JIT, tailcalls remain unchecked. Flow Hijacking can therefore exploit the Tailcall Trampoline technique to hijack the JIT execution flow and achieve malicious behavior. However, despite the possibility of exploiting the Tailcall Trampoline in JIT, launching a practical attack remains challenging.

First, injecting malicious code and redirecting execution to it is no longer feasible in the eBPF JIT context. Unlike attacks targeting the eBPF interpreter, Flow Hijacking against the JIT cannot simply insert malicious JITed (native) code and jump to it because of protections, such as W⊕X and SMEP. All JITed native code resides in the read-only pages after JIT compilation. Moreover, SMEP prevents the kernel from executing user-space code. As a result, we cannot inject native kernel code. Second, traditional code reuse attack techniques are also ineffective. Flow Hijacking corrupts only a single tailcall and offers almost no control over the stack or register state, falling short of the requirements for traditional code reuse attacks such as ROP or JOP. Moreover, kernel CFI further blocks such techniques, rendering them impractical in this context.

To overcome these challenges, we introduce a novel exploitation technique for attacking the eBPF JIT, named *JIT-Type Confusion*. This technique leverages the Tailcall Trampoline to skip the legitimate value-loading code, thereby tricking eBPF into interpreting a malicious numeric value as a pointer and achieving type confusion. JIT-Type Confusion enables arbitrary kernel memory access, paving the way for further attacks, including data leakages and privilege escalation attacks.

5.2 JIT-Type Confusion

To launch JIT-Type Confusion, we conduct an in-depth analysis of the eBPF tailcall implementation and discover that certain eBPF registers remain untouched during the tailcall process. This property allows the values in those registers to persist while their types can be corrupted.

According to the eBPF Instruction Set Architecture (ISA) [21], registers `BPF_REG_1` through `BPF_REG_5` serve as arguments for function calls. In practice, tailcall stubs consume only `BPF_REG_1`–`BPF_REG_3` (and on `x86`, `BPF_REG_4` for bookkeeping). As a result, `BPF_REG_5` is neither read nor overwritten—its value crosses the tailcall boundary unchanged. This register persistence is critical: without it, any attempt to confuse the register's type would require reloading the value from memory after the tailcall, which would trigger fresh verifier checks and prevent type confusion.

To exploit `BPF_REG_5`, the attacker first legally constructs numeric data assigned to this register. Then, the attacker prepares instructions to reload a pointer type into the same register. By leveraging the Tailcall Trampoline to skip these reload instructions, the numeric data is subsequently treated as a pointer, without violating kernel CFI protections or triggering eBPF verifier checks.

Attacking steps. Let's use Figure 4 to illustrate the attacking steps of JIT-Type Confusion. The initial steps—creating a `prog_array` and corrupting its entries to hijack a tailcall—are identical to the interpreter attack described in Section 4.2 and are omitted here for brevity. To trigger JIT-Type Confusion, the attacker first allocates a non-`prog_array` map (specifically, a `BPF_MAP_TYPE_ARRAY`) to exchange attacker-controlled data with the eBPF program ❶. Next, the attacker crafts two eBPF programs via `sys_bpf` ❷: The main program (Lines 1-7) reads an attacker-chosen value from the data array into register `BPF_REG_5` ❸ (Line 3) and then performs a tailcall to the "sub" program (Line 7). The sub-program (Lines 9-14) is carefully constructed, beginning with two code lines, which load `BPF_REG_5` with a legitimate map pointer and then dereference it for read or write operations (Lines 10-11). Up to this point, all dereferences are safe because these two lines correctly initialize a valid pointer before any memory access.

However, after hijacking the tailcall, the control flow skips the initialization instructions (Lines 10-11) and jumps directly into Lines 13-14 (highlighted with a red background)

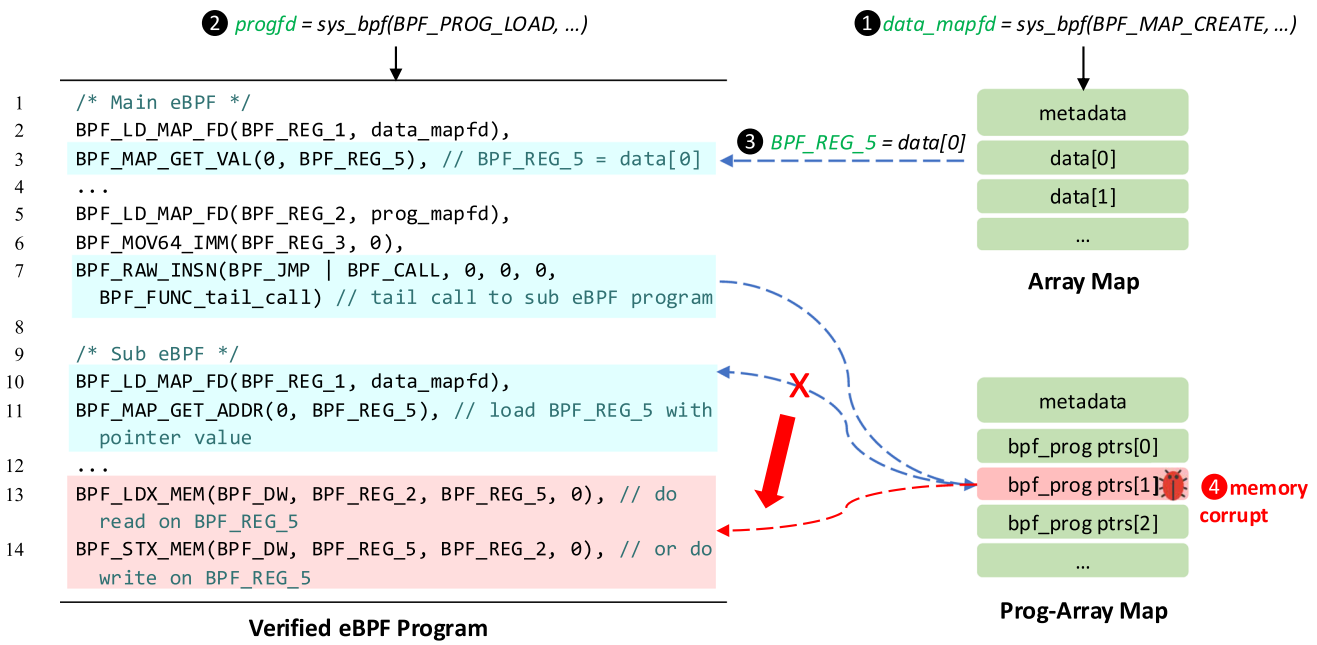


Fig. 4 The basic idea and attacking steps of JIT-Type Confusion. The core is to break the verifier's type inferences via a corrupted tailcall. Steps ❶ ❷ ❸ don't require any privileges or capabilities

❹. As noted earlier, the value in `BPF_REG_5` persists across the tailcall boundary without modification. Consequently, the attacker-controlled data loaded into `BPF_REG_5` in Line 3 is treated at runtime as a valid kernel pointer.

Through these steps, JIT-Type Confusion effectively transforms attacker-controlled data into a pointer type, enabling arbitrary memory access without requiring additional privileges (loading `SOCKET_FILTER` programs and creating an array map doesn't require any capabilities or privileges, as discussed in Section 3.1). Building on JIT-Type Confusion, an attacker can further escalate privileges to gain even more powerful access within the system.

6 Exploitability Evaluation

In this section, we evaluate the exploitability of Flow Hijacking against real-world CVEs. We first define the capabilities a CVE must provide to enable Flow Hijacking and then show how to pivot general kernel CVEs to satisfy these requirements. Finally, we give a list of CVEs that our attack can leverage.

6.1 CVE Capability Requirement

As mentioned in Section 4.2, Tailcall Trampoline depends on corrupting a heap-allocated prog-array map. Therefore, for any memory corruption vulnerability, such as Double-Free (DF), Out-Of-Bounds (OOB), or Use-After-Free (UAF), as long as it can achieve the prog-array map corruption, it can exploit Flow Hijacking to bypass kernel protection and attack the kernel. Moreover, since the prog-array's size (from 512B to 8KB) is chosen by the user, an attacker can influence its slab allocation, expanding the set of exploitable CVEs.

6.1.1 CVE Exploitation for Interpreter

To provide a clear illustration of the capability requirement, we use CVE-2018-18445 as an example and walk through the entire exploitation process on interpretation settings to show how Flow Hijacking enhances it to achieve privilege escalation. CVE-2018-18445 is an OOB vulnerability caused by incorrect bounds checking, which enables out-of-bound write beyond the eBPF map structures (non-prog-array map). Though it is a vulnerability in eBPF, we use it here for illustration purposes and present how to pivot general kernel CVEs in the subsequent section. The whole attack consists of three steps, as shown in Figure 5.

① **Memory layout manipulation.** First, we need to make the vulnerable object adjacent to the prog-array map so that we can perform an OOB write to corrupt the prog-array map. To achieve this, we create both a BPF array map (the vulnerable target) and a prog-array map of identical size via `sys_bpf(BPF_MAP_CREATE, uattr, size)`, and set `uattr.value_size` and `uattr.max_entries` accordingly. Since neither map requires special privileges, they allocate from the same slab cache. By spraying multiple prog-array maps, it is almost guaranteed that one array map and one prog-array map become adjacent on the heap.

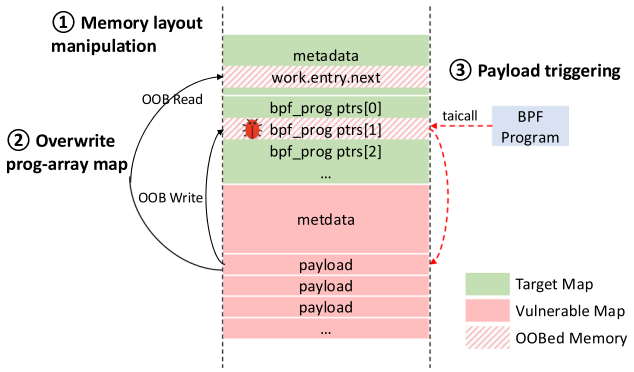


Fig. 5 Memory layout when launching Flow Hijacking with CVE-2018-18445. We OOB read `work.entry.next` field to leak address, and OOB write the `ptrs[1]` to corrupt `bpf_prog` pointer. Payload is injected into the vulnerable map, whose data area is controllable

② **Overwrite prog-array map.** With adjacency achieved, we first OOB read the `work.entry.next` field, which is a self-pointed pointer in the prog-array map, leaking the address of the slab. Moreover, since the data area of the vulnerable map can be specified from user space, we place the payload (malicious bytecode) there. The address of the payload can be inferred from the address of the slab. Next, we OOB write `prog_array->ptrs[1]` with the payload address, redirecting the tailcall target to our injected bytecode.

③ **Payload triggering.** To trigger the payload, we load another `SOCKET_FILTER` program to perform tailcall via `BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0, BPF_FUNC_tail_call)`. We specify the target of tailcall to the corrupted `ptrs[1]` pointed to the payload. Specifically, the interpreter handles `BPF_CALL` with a jump relative to the symbol `__bpf_call_base`, and this offset is written in bytecode. Therefore, we can construct payloads and manipulate the offset in the bytecode to call any kernel function, for example, invoking `commit_creds(prepare_kernel_cred(0))` to escalate privileges.

Note that in the whole process, Tailcall Trampoline does not change kernel code nor violate the CFI protection. Hence, Flow Hijacking can boost the CVE capability to bypass the kernel protection and attack the kernel.

6.1.2 CVE Exploitation for JIT

The capability requirements for exploiting CVEs under JIT execution are identical to those for the interpreter. Below, we highlight only the JIT specific steps, again using CVE-2018-18445 as a concrete example. The whole attack consists of three steps.

① **Memory layout manipulation.** The allocation and manipulation of maps mirror the interpreter case and are omitted here. Additionally, we store an arbitrary 64-bit value v_{att} into the previously created non-prog-array map, and carefully craft two eBPF programs. The "main" program (tailcaller) performs `BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0, BPF_FUNC_map_lookup_elem)` on that map to lookup and move v_{att} into `BPF_REG_5`. Then it tailcalls to the "sub" eBPF program. This "sub" program (tailcallee) mainly contains two basic blocks: The first block performs map lookup again to obtain a valid entry pointer and writes it into `BPF_REG_5`; the second dereferences that point in `BPF_REG_5` to perform the desired read/write.

② **Overwrite prog-array map.** Similar to the interpreter attack, we first utilize the OOB capability to leak the address of the slab and corrupt the prog-array map. The difference lies in that the payload here is not the bytecode we insert into the vulnerable map, but the second block of the "sub" program. Therefore, we OOB write `prog_array->ptrs[1]` to the originally "sub" program address with an offset, thereby allowing the execution to start from the second block. Note that the exact offset is not critical: we can pad the second block with harmless instructions to enlarge the acceptable landing zone.

③ **Payload triggering.** Executing the "main" program now tailcalls into the offset entry of the "sub" program. `BPF_REG_5` still holds v_{att} and remains untouched, but now it's treated as a trusted pointer. The following code dutifully dereferences the register, yielding arbitrary kernel access and paving the way for privilege escalation.

Throughout the whole attack, no new code is injected, kernel text remains immutable, and all indirect branches comply with CFI. Thus, Flow Hijacking boosts the CVE capability to bypass the kernel protection and attack the kernel.

6.2 Pivoting General CVE Capability

The CVEs used for illustration above are from the eBPF subsystem. In this section, we demonstrate that general kernel CVEs from other subsystems can also exploit Flow Hijacking to attack the kernel. In the following, we discuss how to pivot general kernel bugs to achieve the prog-array map corruption capability.

6.2.1 Pivoting Double-free

To exploit double-free CVEs, our basic idea is to free a prog-array map's memory slot via the CVEs and reallocate the same memory slot to an object type whose content can be controlled by the user. In this way, the user can corrupt the prog-array map with any content using the reallocated object, thus allowing the user to redirect the eBPF execution flow.

Let's use Figure 6 to depict the procedure of pivoting a regular double-free (DF) CVE. (a) We trigger the bug to free the vulnerable object and get a freed memory slot. The same slot

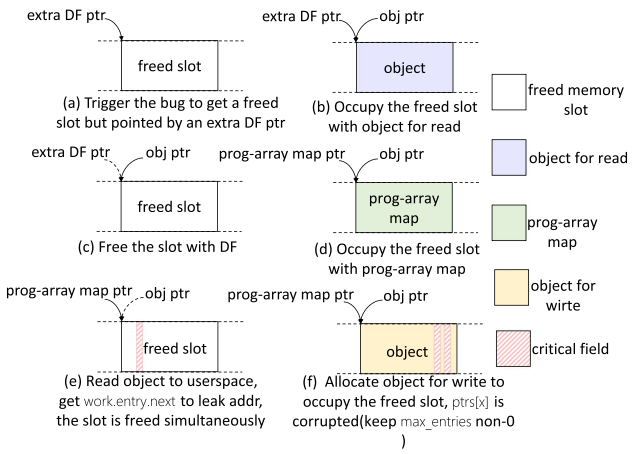


Fig. 6 Steps to pivot general double-free vulnerabilities to launch Flow Hijacking

can be freed again using the DF CVE. (b) We spray objects having the same size with the prog-array map to occupy the freed slot. These objects contain data that is copied to userspace later. (c) We free the object again, with DF capability. (d) We spray the prog-array map to occupy the slot. Such operations replace the data in the object with a prog-array map. (e) We read the objects to userspace via the object allocated in step(b) and get the `work.entry.next` field. In this way, the address of the slab is leaked. The kernel frees the slot simultaneously. (f) We spray objects again to occupy the slot. Users can specify the content of these objects, so we can corrupt the prog-array map. In the interpreter attack case, since five instructions (40 bytes) are enough to achieve privilege escalation, the payload is small enough to be deployed in the object or the adjacent one. Regarding the JIT attack, no payload is required.

After doing all the above steps, a DF CVE can successfully exploit Tailcall Trampoline to redirect the eBPF execution flow to malicious paths, bypassing kernel code or CFI protection.

6.2.2 Pivoting OOB and UAF

A normal out-of-bounds (OOB) or use-after-free (UAF) CVE may not be able to corrupt the `ptrs` field of prog-array map directly, since they usually have a limited corruption range. For example, some OOB bugs can only corrupt several bytes out of range, and UAF bugs can only write to a specific offset within the object. While it's difficult to directly exploit OOB or UAF CVEs, in turn, we pivot them to double-free and use the DF capability to chain the attack, as described above.

Our basic idea is to corrupt one of the pointers pointed to sprayed objects (the same size as the prog-array map) with these bugs, making two pointers point to the object, achieving the double-free capability. Specifically, we first spray objects having the same size as the prog-array map into the slab. There's a good chance they're in the same cache. The pointers pointed to those objects are kept in other objects, and we select them as intermediate target objects. Here we use `msg_msg` as the intermediate target object, which contains a pointer pointing to another `msg_msg` struct (sprayed objects) in its first 8 bytes. Both `msg_msg` sizes can be adjusted from 64B to 4KB. Second, we trigger the vulnerability to overwrite the low bytes of a pointer in the intermediate target objects. For example, we can simply corrupt the low bytes to zero. In this way, there is a sprayed object pointed to by both the original pointer and the corrupted one. And we can use these two pointers to free the object twice, achieving a double-free. And finally, we can launch Flow Hijacking with this converted double-free as presented in Section 6.2.1.

In our evaluation using real-world CVEs (Table 2), we have observed that the utilization of intermediate objects (i.e., `msg_msg`) can greatly enhance the capabilities of weak OOB and UAF CVEs. This improvement is particularly useful for vulnerabilities that are limited to corrupting only the first one or two bytes out of bounds.

6.3 CVE Summary

We evaluate the exploitability of Flow Hijacking using 16 real-world CVEs. Specifically, we select 7 CVEs from the eBPF subsystem and 9 CVEs from other subsystems of the kernel to demonstrate the generality. The results show that Flow Hijacking can enhance all their capabilities to bypass kernel protection to achieve arbitrary bytecode execution, arbitrary memory access, or privilege escalation, as summarized in Table 2.

Selection of CVEs. Flow Hijacking can be applied to any CVEs if they can corrupt the prog-array map on the heap. Moreover, since the user can control the length of the prog-array map ranging from 512B to 8KB, Flow Hijacking can be exploited by more vulnerabilities. All CVEs in the eBPF subsystem meeting this requirement can exploit Flow Hijacking in a similar approach as CVE-2018-18445, as shown in Section 6.1.

For general kernel CVEs, we classify them as: (1) All DF CVEs which arise in 512B-8KB slab cache can be used by Flow Hijacking; (2) Two kinds of OOB CVEs can also be used by Flow Hijacking. First, those CVEs that have `ptrs` of the prog-array map within their corruption range. To be specific, for CVEs OOB to high address, they should OOB write more than `metadata_size+8` bytes; for others OOB to low address, they need to OOB write more than `padding_size+8` bytes. Second, those CVEs can corrupt pointers pointed to the same objects to form a double-free, as discussed in Section 6.2.2. 3) Similar to OOB, two kinds of UAF can be used by Flow Hijacking, including the ones

Table 2 Exploitability evaluation using real-world vulnerabilities. ∗ requires `CAP_BPF`; † exploitable only without CFI. The affected kernel versions are *approximate ranges* aggregated from NVD; finer-grained patch ranges are omitted for brevity

No.	CVE-ID	Type	Affected Kernel Versions
1	CVE-2018-18445	OOB	v4.14–v4.18
2	CVE-2020-8835	OOB	v5.4–v5.6
3	CVE-2020-27194	OOB	v5.0–v5.8
4	CVE-2021−3489*	OOB	v5.8–v5.12
5	CVE-2021-31440	OOB	v5.7–v5.12
6	CVE-2021−34866*	OOB	v5.8–v5.13
7	CVE-2022-23222	OOB	v5.8–v5.16
8	CVE-2021-22555	OOB	v2.6–v5.12
9	CVE-2021-22600	DF	v4.14.x, v4.19.x, v5.4–v5.15
10	CVE-2021-23134	DF/UAF	v5.0–v5.12
11	CVE-2021-26708	UAF	v5.5–v5.10
12	CVE-2021−41073†	UAF	v5.10–v5.14
13	CVE-2022-0185	OOB	v5.1–v5.16
14	CVE-2022-0995	OOB	v5.8–v5.16
15	CVE-2022-2639	OOB	v3.18–v5.17
16	CVE-2022-2588	UAF	v4.9–v5.19

that can write to an offset in the prog-array map within `(metadata_size, metadata_size+8*max_entries)` and the ones that can corrupt pointers to form a double-free.

CVE discussion. Table 2 lists the details of all 16 CVEs that can leverage Flow Hijacking to launch attacks, including CVEs in the eBPF subsystem, and general DF/OOB/UAF CVEs in various subsystems, demonstrating that Flow Hijacking can be applied to general kernel vulnerabilities. Specifically, CVEs in the eBPF subsystem can be exploited with similar approaches to CVE-2018-18445, as shown in Section 6.1. Other CVEs can be pivoted to achieve the capability of corrupting the prog-array map, as described in Section 6.2.

Furthermore, Flow Hijacking significantly enhances the capabilities of all these 16 CVEs. Existing exploits for these CVEs mostly depend on control-flow hijacking, escalating privilege through triggering ROP chains. To the contrary, Flow Hijacking utilizes Tailcall Trampoline to give these CVEs the ability to bypass CFI. This feature makes Flow Hijacking a more practical and effective attack, especially considering the widespread deployment of CFI protections.

Note that CVE-2021-41073 targets the pointer in `sk_filter`, a hook point-related data structure. However, eBPF programs pointed by hook point-related structures can only be triggered via invoking `prog->bpf_func`, thus can not bypass CFI. CVE-2021-34866 and CVE-2021-3489 themselves need `CAP_BPF` to be triggered.

7 Turing Completeness Proof

In this section, we will demonstrate the expressiveness of Flow Hijacking when targeting the interpreter.

As discussed in prior works [22–24], the legitimate eBPF program is not Turing-complete, due to the verifier restriction on program size, program termination, memory access, and functionality. However, when targeting the eBPF interpreter, an attacker can leverage Flow Hijacking to execute arbitrary eBPF bytecode while bypassing checks performed by the verifier. In this way, the eBPF bytecode payload can have an unlimited size and contain infinite loops. By constructing appropriate eBPF bytecode, the eBPF bytecode payload can also call kernel functions, such as `vmalloc`, to allocate memory regions to simulate tape memory and store the transition table. Therefore, Flow Hijacking targeting the interpreter can achieve *Turing-completeness*. To prove this, we first define a classical Turing machine. Next, we construct an eBPF program to implement a universal Turing machine simulator capable of arbitrary computation, proving that Flow Hijacking is Turing-complete, in the interpreter case.

7.1 Turing Machine Definition

We leverage Flow Hijacking to construct an attack payload executable in the interpreter to implement a universal Turing machine simulator. Specifically, the payload is an eBPF program that first initializes the Turing machine and then keeps looping to repeatedly simulate the machine steps until the program terminates. A classical Turing machine is defined as follows.

Definition 1 (Specification of a Turing Machine) A (one-tape) Turing machine is a septuple

$$\mathscr{M} \triangleq \left\langle S, s_0, \sigma_0, \Gamma, \Sigma, \bar{S}, \delta \right\rangle \text{, where}$$

- S is a finite, non-empty set of *states*,
- $s_0 \in S$ is the *initial state*,
- Γ is a finite, non-empty set of *symbols* representing alphabets of tape cells,
- $\sigma_0 \in \Sigma$ is a special *blank symbol*,
- $\Sigma \in \Gamma \backslash \{\sigma_0\}$ is a finite, non-empty set of *input symbols* allowed to appear in the initial tape contents,
- $\bar{S} \subseteq S$ is the set of *final states or accepting states*,
- $\delta \subseteq (S \backslash \bar{s}) \times \Gamma \rightharpoonup \Gamma \times \{\mathbb{L}, \mathbb{R}\} \times S$ is a partial function called *transition function* that determines the next move, where $\mathbb{L}$ is left shift and $\mathbb{R}$ is right shift.

Representation. To represent a Turing machine, we set up the data structures in the victim program's memory as follows. $s_{cur} \in S$ holds the current state, and $\mathscr{M}$ stops with a final accepting state if $s_{cur} \in \bar{S}$. We apply the function $\Pi : N \rightarrow \Sigma$ to extract the symbol contained in the tape cell given a certain index; for simplicity, we write Π_i as shorthand for $\Pi(i)$. A pointer `Tape_head` tracks the position on the tape containing the current symbol σ_{cur}, that is, $\sigma_{cur} = \Pi_{\texttt{Tape_head}}$. We employ the transition table as a (partial) consulting function to determine how the Turing machine functions given the current input symbol and state. Specifically, Δ is placed in memory such that the comparison operation to search Δ is transformed into a direct lookup in a two-dimensional array. Given the current state s_{cur} and the current tape symbol σ_{cur}, let $\alpha' = \left(s', \sigma', d\right) \in \Delta$ be the matched table item. Then, $\mathscr{M}$ yields a jump from s_{cur} to s', overwrites σ_{cur} to σ', and moves `Tape_head` in direction $d \in \{\mathbb{L}, \mathbb{R}\}$. $\mathscr{M}$ halts when α' is not find in Δ.

7.2 Turing Machine Initialization and Simulation

Initialization process. In the first step of the attack, an adversary can craft a payload as shown in Figure 7 to load the input symbols $\sigma_0 - \sigma_n$, transition table Δ, and the initial state s_0. The eBPF instructions used to craft the payload in Figure 7 and Figure 8 are collected in Table 3. To achieve this, the attacker first calls the kernel function `vmalloc` to allocate intended-sized memory storing the tape (Line 2) and transition table (Line 8), respectively. This exploits the capability of arbitrary kernel code execution. The start address of the allocated memory is returned in register `R0` by default. Then, the attacker executes the sequence of operations in Figure 7 with eBPF instructions of assignment (`BPF_MOV64_IMM`), dereference (`BPF_STX_MEM`) and addition (`BPF_ADD`). Recall that in the eBPF convention, register R0 stores the return address, R1-R5 pass an argument during function calls, and frame pointer R10 points to a stack memory region. We use the dedicated eBPF registers following the convention and write them explicitly in the pseudo-code. For simplicity, variables with `reg_` prefix are mapped to other registers (`R6-R9`).

```
1  BPF_MOV64_IMM((BPF_REG_1, input_mem_size), //set
   usable memory
2  BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0,
   vmalloc_off_from_call_base) //call vmalloc(MEM_SIZE)
3  BPF_MOV64_REG(reg_tape_head, BPF_REG_0),
4  BPF_ALU64_IMM(BPF_ADD, reg_tape_head, cell_offset)
   //initialize tape head to point to the first cell
5  //write input symbols
6  /***** Initialize transition table Δ *****/
7  BPF_MOV64_IMM((BPF_REG_1, sizeof(Δ)), //set usable
   memory
8  BPF_RAW_INSN(BPF_JMP | BPF_CALL, 0, 0, 0,
   vmalloc_off_from_call_base) //call vmalloc(MEM_SIZE)
9  BPF_MOV64_REG(reg_table_head, BPF_REG_0),
10 //write table items
11 BPF_MOV64_IMM((s_cur, s_0),  //initialize state
```

Fig. 7 eBPF payload to initialize the Turing Machine. We construct eBPF bytecode to input symbols and the transition table, then set the initial state

```
1  BPF_ALU64_IMM(BPF_MUL, s_cur, row_size),
2  BPF_ALU64_REG(BPF_ADD, s_cur, reg_head_2), //get the row
3  BPF_MOV64_REG(reg_temp, s_cur),        //now temp = row ne-step
4  BPF_LDX_MEM(BPF_DW, σ_cur, reg_tape_head, 0),
5  BPF_ALU64_IMM(BPF_MUL, σ_cur, col_size),
6  BPF_ALU64_REG(BPF_ADD, reg_temp, σ_cur), //get the column, now
   temp = col
7  BPF_LDX_MEM(BPF_DW, s_cur, reg_temp, 0), //set the new state
8  // terminate the program if s_cur ∈ S̄
9  BPF_ALU64_IMM(BPF_ADD, reg_temp, state_size),
10 BPF_LDX_MEM(BPF_DW, σ_cur, reg_temp, 0), //read symbol
11 BPF_STX_MEM(DW, σ_cur, reg_tape_head, 0), //update symbol
12 BPF_ALU64_IMM(BPF_ADD, reg_temp, symbol_size),
13 BPF_LDX_MEM(BPF_DW, reg_temp, reg_temp, 0),
14 BPF_ALU64_REG(BPF_ADD, reg_tape_head, reg_temp),//move the head
15 BPF_JMP_IMM(BPF_JA, 0, 0, simulation_off),//return to the
   beginning
```

Fig. 8 eBPF payload to simulate one step in the Turing Machine. It queries the transition table with the current state and sets the new one. We use JMP to construct the loop and EXIT to halt the Turing machine

Table 3 eBPF instructions used to simulate $\mathscr{M}$ operations. We use ALU, MEM, and CALL in initialization, ALU, MEM, JMP, and EXIT in simulating one execution step

Instruction	Category	Meaning
`BPF_RAW_INSN`	CALL	call to a kernel function
`BPF_LDX_MEM`	MEM	atomic exchange-and-add
`BPF_STX_MEM`		
`BPF_ADD`	ALU	arithmetic and bitwise
`BPF_MUL`		
`BPF_EXIT_INSN`	EXIT	terminate
`BPF_JMP_IMM`	JMP	(un)conditional jump

Simulating one step.

After initialization, the attacker further constructs a payload as shown in Figure 8 that simulates machine steps repeatedly until the program terminates. Recall that each Turing machine step queries the transition table Δ based on the current symbol σ_{cur} and state s_{cur}, and then updates s_{cur}, σ_{cur}, and `Tape_head` accordingly. We use the addition (`BPF_ADD`), multiplication (`BPF_MUL`), and load (`BPF_LDX_MEM`) operations to dynamically calculate the offset in Δ (Line 1-6) and look up the next state and symbol. Note that references to `Tape_head`, Δ, and s_{cur} are internally stored in the eBPF program's stack, and are loaded to registers like `reg_head_2`. Next, we use the addition and store (`BPF_STX_MEM`) operations to update s_{cur} (Line 7), write the new fetched symbol (Line 11), and move `Tape_head` (Line 14). In particular, $s_{cur} \in \bar{S}$ indicates the termination of the Turing machine, and thereby the program exits (`BPF_EXIT_INSN`). Otherwise, the CPU stack pointer will be redirected to the top of the payload loop with a jump operation (`BPF_JA`, Line 15).

8 Mitigation

To defend against Flow Hijacking, we add a runtime check that ensures that only verified eBPF programs will be executed. The basic idea is to verify whether the `bpf_prog` pointer is legitimate whenever it is used, either by the interpreter or JIT, by searching the set of verified `bpf_prog` structures.

8.1 Design

One key challenge is to determine whether a `bpf_prog` pointer is legitimate or not, as the kernel does not authenticate pointers before using them. To overcome this, we propose IDR match-based verification. We observe that the Linux kernel already keeps a radix-tree (`prog_idr`) that indexes every verified eBPF program by a unique ID. Therefore, we can identify a forged `bpf_prog` if its id is not in the radix tree or the pointer value differs from the tree entry. Specifically, we search the eBPF program to be executed in the verified set with its ID as the key. As the verified eBPF program is organized as a radix tree, the time complexity of the search operation is constant, resulting in minimal performance overhead.

On the other hand, we need to ensure that the check mediates all paths in which the eBPF program may be executed. Recall that the kernel executes all eBPF programs via a specific entry function, so we set checkpoints at that entry. Besides, some eBPF programs can also be triggered via tailcall invoked from other eBPF, so a checkpoint should be set before the eBPF tailcall occurs. By performing checks on the target `bpf_prog` at the entry and before the tailcall, Tailcall Trampoline is defeated, thereby mitigating Flow Hijacking across both paths.

8.2 Implementation

We implemented the proposed defense against Flow Hijacking on Linux v6.1.33.

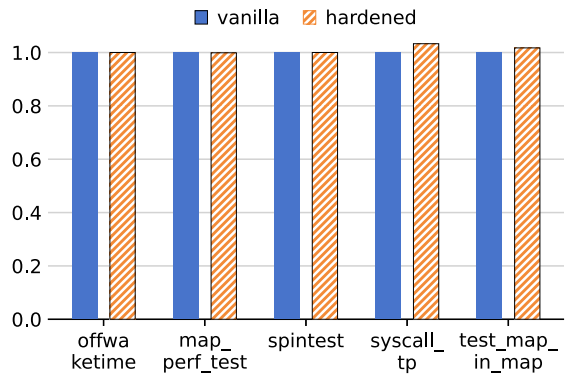
IDR-based Verification. The kernel maintains a `prog_idr` structure to support efficient query and update on `bpf_prog`. To validate an eBPF program, we first obtain its program pointer (`prog`) from the entry function's parameters. Next, we acquire its program ID by reading the `prog->aux->id` field. We then use the `id` as the key to query the `prog_idr` via invoking `idr_find`. This retrieves the pointer of the legitimate program assigned to this `id`. Lastly, we compare the pointer of the supplied program and the legitimate program. If the addresses match, the supplied program is considered legitimate.

Checkpoints for Interpreter. We add the above check at the entry of the interpreter and the tailcall handling branch. In the Linux kernel, the execution of eBPF is triggered through an entry function called `bpf_prog_run`, which will dispatch the eBPF program with its `bpf_func`. As we introduced in Section 2, `bpf_func` will be either the interpreter or the JITed native code. This function serves as the entry for both the interpreter and JIT, and we modify it to add our check at the beginning. In regard to tailcall check, we modify the `JMP_TAIL_CALL` branch in the interpreter. In this branch, the kernel looks up the prog-array map with the index as key and gets the target prog to be tailcalled. We check this target prog before the kernel accesses its `insnsi` field to get the bytecode.

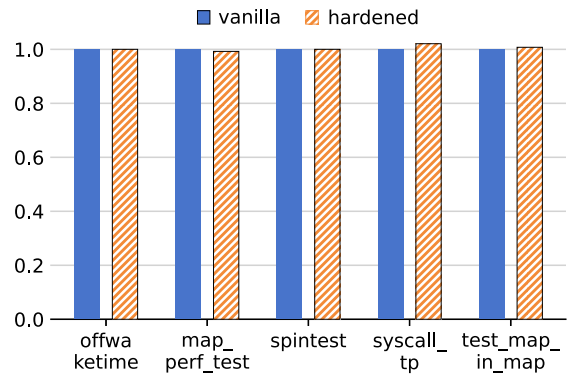
Checkpoints for JIT. For the JIT path, we propose modifying the JIT compiler, specifically the `emit_bpf_tail_call_indirect` function, to call our verification function before performing the tailcall to eBPF subprograms.

8.3 Performance Evaluation

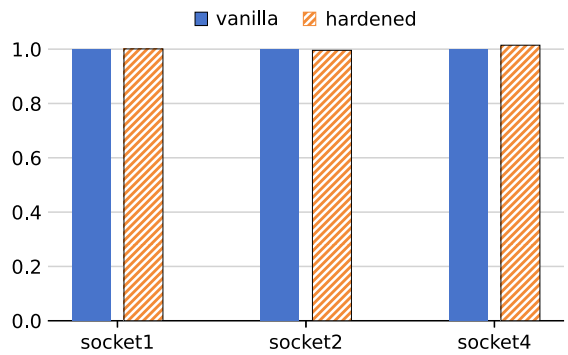
To evaluate the overhead of our defense, we run eBPF sample programs (for throughput and latency) from the Linux source tree on the vanilla kernel and hardened kernel of v6.1.33. Specifically, to simulate the scenario of socket filtering, we use a network tool to continuously send TCP packets with a body size of 64 bytes to our machine. We attach a raw socket to the Ethernet interface and attach an eBPF program on the raw socket to filter the packets coming through the socket, and measure the running time until the number of received and processed packets reaches 10^{10}. The evaluations are conducted on a Dell OptiPlex 7070 workstation with a



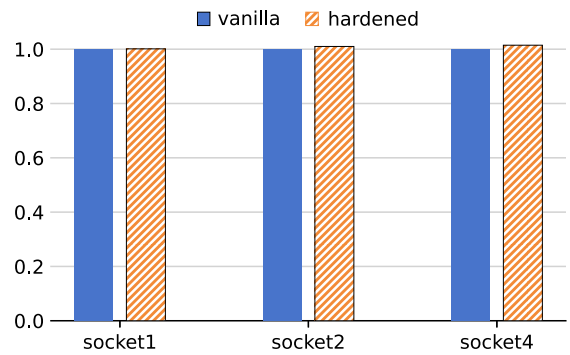
(a) Normalized latency overhead in Interpreter mitigation (Avg 0.9%)



(c) Normalized latency overhead in JIT mitigation (Avg 0.5%)



(b) Normalized throughput overhead in Interpreter mitigation (Avg 0.5%)



(d) Normalized throughput overhead in JIT mitigation (Avg 0.8%)

Fig. 9 Normalized overhead of IDR-based mitigation in both Interpreter and JIT execution

6-core Intel i7-8700 CPU, 32GB of memory, and a 1000GB SSD. The workstation is provisioned with Ubuntu 22.04.3 LTS; the interpreter-path measurements run with the BPF JIT disabled, while the JIT-path measurements run with the BPF JIT enabled on the same kernel and host.

Figure 9 shows that the average performance overhead introduced by our defense is 0.9%, 0.5% in the interpreter path, and 0.5%, 0.8% in the JIT path, respectively. Note that our defense incurs less than 1% overhead in most cases.

9 Limitation and Discussion

As we discussed in Section 3.1, though tailcall is not protected by CFI, IBT protections still pose a barrier to our current JIT attack. Here we discuss a bypass method with a BPF-to-BPF call, which requires `CAP_BPF`. Note that the IBT-on bypass below requires `CAP_BPF` and is outside our unprivileged attacker model in Section 3.1.

The core of Flow Hijacking targeting JIT execution lies in splitting the exploitation code into: (i) Type preparation: initialize a register (e.g., `BPF_REG_5`) with a valid pointer so the verifier's static analysis passes; (ii) Utilization: perform memory reads/writes by dereferencing that register. A hijacked tailcall into the second stage can achieve arbitrary memory access, but would violate IBT, since it jumps into the middle of a function.

However, a BPF-to-BPF call can bypass this. Specifically, we can separately split these two stages into two functions. The first function performs the verifier-approved pointer load, then invokes the second via a BPF-to-BPF call, passing the register value as a parameter. The verifier does not re-check parameter types on such calls, so the second function executes with the attacker's data treated as a pointer. By hijacking the tailcall to land at the entry of this second function, which serves as a legitimate indirect call target under IBT, we evade those protections.

10 Related Work

10.1 eBPF Security

EPF [18] and Ret2bpf [10] [25] are the most closely related works to ours. They use bytecode to facilitate traditional control-flow hijacking attacks, essentially, which can be blocked by CFI and are out of the definition of our flow hijacking.

Securing the verifier and JIT compiler. Due to the complexity of eBPF subsystem, many bugs related to its core components, including the verifier and JIT compilers, are exposed [26, 27]. Researchers make efforts to deal with these potential risks. Lu et al. propose MOAT [16] to isolate potentially malicious eBPF programs at runtime. Since MOAT utilizes Intel Memory Protection Keys (MPK) to authenticate accessible memory space for eBPF programs, it can only support at most 16 memory regions concurrently. Google initiated Buzzer [5] project, which targets validating the eBPF verifier with fuzzing. Buzzer generates random eBPF programs and then checks whether they have unexpected behavior. Gershuni et al. propose an improved verifier [8] to support better asymptotic complexity without compromising on safety. Vishwanathan et al. proved the abstract arithmetic operations used for addition and subtraction of them in the eBPF verifier formally [9]. Nelson et al. propose Jitterbug [28] to formally verify eBPF JIT compilers and target the implementation of them correctly.

However, these works overlook the intrinsic risks introduced by the vulnerable workflow design of eBPF programs, which are only checked once at the load stage and have few restrictions during the execution stage, allowing attackers to bypass the verifier by exploiting kernel vulnerabilities. We launch Flow Hijacking based on such observation.

eBPF exploitability. Researchers propose to utilize eBPF bytecode to overwrite target objects (such as credential structs) [10, 11, 25]. The kernel enforces extra protection on these target objects, which contain sensitive data, so they cannot be corrupted via common kernel bugs but require eBPF's facilitation. However, these attacks need to either corrupt or forge a kernel function pointer, as CFI has been widely adopted in the kernel; therefore, such attacks are defeated. On the other hand, they only use eBPF bytecode to write target objects without demonstrating its powerful expressiveness that can actually execute arbitrary kernel code.

Researchers also abused eBPF to deploy a backdoor and rootkit [29–32]. Kernel developers implemented capability-based, finer-grained access control for eBPF operations. Deploying such eBPF-based malware needs high privilege, e.g., `CAP_SYS_ADMIN` first. Therefore, attackers usually utilize eBPF to leave a backdoor after escalating privileges as root. In contrast, Flow Hijacking can be launched by a non-privileged user without any capability at all.

10.2 Common Kernel Attacks

Code injection attack. Code injection is a common exploitation method used by attackers. W⊕X mitigation is proposed to defeat the shellcode injection. However, attackers find other approaches to inject shellcode, a popular one is ret2usr [33], which pivots the kernel to execute userspace code. After SMEP [34] is deployed, ret2usr is resisted, combining SMAP [35] to strengthen the isolation further between kernel space and user space. However, ret2dir [36] is proposed by Kemerlis et al, it utilizes `physmap` to map synonyms of user memory to kernel space. Both user-controllable code and data can be injected into the kernel in this way, bypassing SMEP and SMAP. To mitigate ret2dir, the kernel has restricted non-privileged users from accessing page map information and managed direct mapping memory more carefully. In short, with kernel code protection enabled, it's difficult for attackers to inject machine code into kernel space. Unlike the shellcode injection techniques based on machine code, Flow Hijacking can bypass all these defenses, because it either injects bytecode as normal kernel data in the interpreter execution, or just reuses the verified JITed code.

Code reuse attack. Attackers turn to code reuse attacks due to the enforced protection mechanisms of code injection. Tran et al. designed a return-to-libc (RILC) attack [37]. Moreover, they demonstrate that RILC attacks are indeed capable of enabling arbitrary computation, rather than merely possessing limited expressive power. Wu et al. introduce KPELER [38] that can facilitate automatic exploit generation with a control-flow hijacking primitive acquired. It exploits the primitive by converting it into a classic stack overflow, utilizing specific gadgets in the kernel. These attacks rely on control flow hijacking and target control data. However, researchers have proposed practical implementations [39–42]. And CFI [43] has been widely adopted in the kernel, preventing control flow hijacking. Our core technique Tail-call Trampoline currently is not protected by CFI.

Non-control data attack. Exploitation methods based on control data are straightforward but less feasible, so non-control data attacks [44–47] are proposed. These attacks manipulate non-control data to affect a program's benign behavior without violating its CFI protection. Lin et al. propose a novel technique called Dirtycred [19] that replaces a low-privileged credential with a high-privileged one to bypass permission checks. Researchers try to figure out how to launch complicated non-control data attacks with their limited expressive power. Some researchers propose techniques to provide Turing-complete computation by exploiting auxiliary features in software [48–51]. And Hu et al. present

a programming paradigm called "data-oriented programming"(DOP) [52] and study the expressiveness of non-control data attacks. Their evaluation indicates that DOP can build Turing-complete attacks. However, DOP still relies on finding and chaining gadgets existing in programs, and it selects only userspace programs as research objects. Relatively, Flow Hijacking does not rely on gadgets in the kernel itself and can inject the desired payload or reuse the verified eBPF program instead.

11 Conclusion

This paper systematically investigates the exploitability of eBPF and uncovers previously unidentified security risks: the eBPF interpreter lacks defenses against hijacking, and its JIT compiler's CFI enforcement is incomplete. To understand the risk, we propose a novel attack named *Flow Hijacking*, which can bypass kernel code and CFI protections, enabling arbitrary memory access and kernel code execution. To launch Flow Hijacking, we propose two key techniques: *Tailcall Trampoline* corrupts tailcall to divert the interpreter to injected code, whereas *JIT-Type Confusion* forges a trusted pointer in JITed code. We further evaluate Flow Hijacking using 16 real-world CVEs from different subsystems of the Linux kernel, demonstrating that Flow Hijacking can enhance all their capability to bypass kernel protections and attack the kernel. We also present a formal proof that Flow Hijacking attains Turing completeness when targeting the interpreter. Finally, we design and implement a runtime validation mechanism within the eBPF interpreter and JIT, effectively mitigating Flow Hijacking attacks across both paths with less than 1% overhead, thereby improving the overall security of both eBPF and the Linux kernel.

Author Contributions Yifei Wu conceived and implemented the JIT-path attack, performed the associated analysis and mitigation, and drafted the manuscript. Qirui Liu originated the interpreter-path attack, designed its defenses, and co-designed the overall study. Zhuoruo Zhang and Rui Chang formalized the Turing-completeness analysis of interpreter attacks. Wenbo Shen contributed to the overall manuscript text and substantive revisions. Jiayi Hu assisted with the evaluations across real-world CVEs. All authors approved the final manuscript.

Funding This work is supported by the National Key R&D Program of China (2022YFB3103900).

Data Availability This work does not generate or analyze any datasets.

Declarations

Competing interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Calavera, D., Fontana, L.: Linux Observability with BPF: Advanced Programming for Performance Analysis and Networking. O'Reilly Media (2019)
2. Miano, S., Bertrone, M., Risso, F., Tumolo, M., Bernal, M.V.: Creating complex network services with ebpf: Experience and lessons learned. In: 2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR), pp. 1–8 (2018). https://doi.org/10.1109/HPSR.2018.8850758
3. Sysdig: Prometheus. https://sysdig.com/opensource/prometheus/ (2023). Accessed 27 June 2024
4. Sysdig: Threat detection built on falco. https://sysdig.com/opensource/falco/ (2016). Accessed 27 June 2024
5. Google: Buzzer - an ebpf fuzzer toolchain. https://github.com/google/buzzer (2023). Accessed 27 June 2024
6. Luke, Nelson, X., Wang, E., Torlak: A proof-carrying approach to building correct and flexible in-kernel verifiers. https://homes.cs.washington.edu/~lukenels/slides/2021-09-23-lpc21.pdf (2021). Accessed 27 June 2024
7. Wang, X., Lazar, D., Zeldovich, N., Chlipala, A., Tatlock, Z.: Jitk: A trustworthy in-kernel interpreter infrastructure. In: 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14), pp. 33–47 (2014)
8. Gershuni, E., Amit, N., Gurfinkel, A., Narodytska, N., Navas, J.A., Rinetzky, N., Ryzhyk, L., Sagiv, M.: Simple and precise static analysis of untrusted linux kernel extensions. In: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 1069–1084 (2019). https://doi.org/10.1145/3314221.3314590
9. Vishwanathan, H., Shachnai, M., Narayana, S., Nagarakatte, S.: Sound, precise, and fast abstract interpretation with tristate numbers. In: 2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 254–265. IEEE (2022). https://doi.org/10.1109/CGO53902.2022.9741267
10. Xingyu, Jin, R., Neal: The art of exploiting uaf by ret2bpf in android kernel. https://www.blackhat.com/eu-21/briefings/schedule/#the-art-of-exploiting-uaf-by-retbpf-in-android-kernel-24544 (2021). Accessed 27 June 2024
11. chompie1337: chompie1337/linux_lpe_io_uring_cve-2021-41073. https://github.com/chompie1337/Linux_LPE_io_uring_CVE-2021-41073 (2022). Accessed 27 June 2024
12. Elena, Reshetova, F., Bonazzi, N., Asokan: Randomization can't stop bpf jit spray. https://www.blackhat.com/eu-16/briefings/schedule/#randomization-cant-stop-bpf-jit-spray-4516 (2016). Accessed 27 June 2024
13. Samsung: Hardening hostile code in ebpf. https://samsung.github.io/kspp-study/bpf.html (2023). Accessed 27 June 2024
14. Starovoitov, A.: [patch v7 bpf-next 0/3]. https://lore.kernel.org/bpf/6f56ba3e-144f-29be-c35d-0506fe16830f@iogearbox.net/T/. Accessed 27 June 2024
15. Dileo, J.: Evil ebpf: Practical abuses of an in-kernel bytecode runtime. DEF CON (2019)
16. Lu, H., Wang, S., Wu, Y., He, W., Zhang, F.: Moat: Towards safe bpf kernel extension. arXiv preprint arXiv:2301.13421 (2023)
17. Azab, A.M., Ning, P., Shah, J., Chen, Q., Bhutkar, R., Ganesh, G., Ma, J., Shen, W.: Hypervision across worlds: Real-time kernel protection from the arm trustzone secure world. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, pp. 90–102 (2014). https://doi.org/10.1145/2660267.2660350
18. Jin, D., Atlidakis, V., Kemerlis, V.P.: {EPF}: Evil packet filter. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23), pp. 735–751 (2023)

19. Lin, Z., Wu, Y., Xing, X.: Dirtycred: Escalating privilege in linux kernel. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, pp. 1963–1976 (2022). https://doi.org/10.1145/3548606.3560585
20. Righi, A.: temporarily disable config_x86_kernel_ibt (2022). https://bugs.launchpad.net/ubuntu/+source/linux/+bug/1980484. Accessed 13 May 2025
21. The Linux Kernel Community: BPF ABI Recommended Conventions and Guidelines v1.0. https://www.kernel.org/doc/html/latest/bpf/standardization/abi.html#id1. Accessed 13 May 2025
22. Amit, N., Wei, M.: The design and implementation of hyperupcalls. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18), pp. 97–112. USENIX Association, Boston, MA (2018). https://www.usenix.org/conference/atc18/presentation/amit
23. Arras, P.A., Andronidis, A., Pina, L., Mituzas, K., Shu, Q., Grumberg, D., Cadar, C.: Sabre: load-time selective binary rewriting. Int. J. Softw. Tools Technol. Transfer **24**(2), 205–223 (2022). https://doi.org/10.1007/s10009-021-00644-w
24. Miano, S., Bertrone, M., Risso, F., Bernal, M.V., Lu, Y., Pi, J., Shaikh, A.: A service-agnostic software framework for fast and efficient in-kernel network services. In: 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS), pp. 1–9 (2019). https://doi.org/10.1109/ANCS.2019.8901880
25. Azad, B.: Project zero: An ios hacker tries android. https://googleprojectzero.blogspot.com/2020/12/an-ios-hacker-tries-android.html (2020). Accessed 27 June 2024
26. MITRE, C.: Cve - cve-2021-29154. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-29154 (2021). Accessed 27 June 2024
27. MITRE, C.: Cve - cve-2021-3490. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-3490 (2021). Accessed 27 June 2024
28. Nelson, L., Van Geffen, J., Torlak, E., Wang, X.: Specification and verification in the field: Applying formal methods to bpf just-in-time compilers in the linux kernel. In: Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation, pp. 41–61 (2020)
29. Dileo, J.: Evil ebpf in-depth: Practical abuses of an in-kernel bytecode runtime. https://defcon.org/html/defcon-27/dc-27-speakers.html#Dileo (2019). Accessed 27 June 2024
30. Guillaume Fournier, S.A., Baubeau, S.: ebpf, i thought we were friends ! https://defcon.org/html/defcon-29/dc-29-speakers.html#fournier (2021). Accessed 27 June 2024
31. PatH: Warping reality - creating and countering the next generation of linux rootkits using ebpf. https://defcon.org/html/defcon-29/dc-29-speakers.html#path (2021). Accessed 27 June 2024
32. Guillaume Fournier, S.A., Baubeau, S.: With friends like ebpf, who needs enemies? https://www.blackhat.com/us-21/briefings/schedule/#with-friends-like-ebpf-who-needs-enemies-23619 (2021). Accessed 27 June 2024
33. Kemerlis, V.P., Portokalidis, G., Keromytis, A.D.: kguard: Lightweight kernel protection against return-to-user attacks. In: USENIX Security Symposium, vol. 16 (2012)
34. Jurczyk, M., Coldwind, G.: Smep: What is it, and how to beat it on windows. https://j00ru.vexillium.org/2011/06/smep-what-is-it-and-how-to-beat-it-on-windows (2011). Accessed 27 June 2024
35. Corbet, J.: Supervisor mode access prevention [lwn.net]. https://lwn.net/Articles/517475/ (2012). Accessed 27 June 2024
36. Kemerlis, V.P., Polychronakis, M., Keromytis, A.D.: ret2dir: Rethinking kernel isolation. In: 23rd USENIX Security Symposium (USENIX Security 14), pp. 957–972 (2014)
37. Tran, M., Etheridge, M., Bletsch, T., Jiang, X., Freeh, V., Ning, P.: On the expressiveness of return-into-libc attacks. In: Recent Advances in Intrusion Detection: 14th International Symposium, RAID 2011, Menlo Park, CA, USA, September 20-21, 2011. Proceedings 14, pp. 121–141. Springer (2011). https://doi.org/10.1007/978-3-642-23644-0_7
38. Wu, W., Chen, Y., Xing, X., Zou, W.: Kepler: Facilitating control-flow hijacking primitive evaluation for linux kernel vulnerabilities. In: USENIX Security Symposium, pp. 1187–1204 (2019)
39. Criswell, J., Dautenhahn, N., Adve, V.: Kcofi: Complete control-flow integrity for commodity operating system kernels. In: 2014 IEEE symposium on security and privacy, pp. 292–307. IEEE (2014).https://doi.org/10.1109/SP.2014.26
40. Edge, J.: Control-flow integrity for the kernel [lwn.net]. https://lwn.net/Articles/810077/ (2020). Accessed 27 June 2024
41. Ge, X., Talele, N., Payer, M., Jaeger, T.: Fine-grained control-flow integrity for kernel software. In: 2016 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 179–194. IEEE (2016). https://doi.org/10.1109/EuroSP.2016.24
42. Yoo, S., Park, J., Kim, S., Kim, Y., Kim, T.: In-kernel control-flow integrity on commodity oses using arm pointer authentication. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 89–106 (2022)
43. Google: Kernel control flow integrity. https://source.android.com/docs/security/test/kcfi (2022). Accessed 27 June 2024
44. Chen, S., Xu, J., Sezer, E.C., Gauriar, P., Iyer, R.K.: Non-control-data attacks are realistic threats. In: USENIX security symposium **5**, 146 (2005)
45. Cheng, L., Ahmed, S., Liljestrand, H., Nyman, T., Cai, H., Jaeger, T., Asokan, N., Yao, D.: Exploitation techniques for data-oriented attacks with existing and potential defense approaches. ACM Transactions on Privacy and Security (TOPS) **24**(4), 1–36 (2021). https://doi.org/10.1145/3462699
46. Hu, H., Chua, Z.L., Adrian, S., Saxena, P., Liang, Z.: Automatic generation of data-oriented exploits. In: 24th USENIX Security Symposium (USENIX Security 15), pp. 177–192 (2015)
47. Carlini, N., Barresi, A., Payer, M., Wagner, D., Gross, T.R.: Control-flow bending: On the effectiveness of control-flow integrity. In: 24th USENIX Security Symposium (USENIX Security 15), pp. 161–176 (2015)
48. Shapiro, R., Bratus, S., Smith, S.W.: "weird machines" in elf: A spotlight on the underappreciated metadata. In: 7th USENIX Workshop on Offensive Technologies (WOOT 13) (2013)
49. Oakley, J.: Exploiting the hard-working dwarf: Trojan and exploit techniques with no native executable code. In: 5th USENIX Workshop on Offensive Technologies (WOOT 11) (2011)
50. Bangert, J., Bratus, S., Shapiro, R., Smith, S.W.: The page-fault weird machine: Lessons in instruction-less computation. In: 7th USENIX Workshop on Offensive Technologies ({WOOT} 13) (2013)
51. Rushanan, M., Checkoway, S.: Run-dma. In: 9th USENIX Workshop on Offensive Technologies (WOOT 15) (2015)
52. Hu, H., Shinde, S., Adrian, S., Chua, Z.L., Saxena, P., Liang, Z.: Data-oriented programming: On the expressiveness of non-control data attacks. In: 2016 IEEE Symposium on Security and Privacy (SP), pp. 969–986. IEEE (2016). https://doi.org/10.1109/SP.2016.62

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.